15.2 Agent Loop（Agent 循环）端到端解析

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~350,000 tokens，输出 ~28,000 tokens（本章合计）

一条用户消息经过通道和路由最终需要 AI 回复时，它会触发一次完整的 **Agent Loop**（Agent 循环）。这个循环是 OpenClaw 里最复杂的流程之一，涉及参数校验、模型解析、凭据准备、队列序列化、LLM 调用、错误恢复等多个阶段。本节就端到端跟踪一条消息从进入 Gateway 到产生 AI 回复的完整路径。

15.2.1 循环入口：Gateway 的 `agent` RPC 方法

Agent Loop 的入口是 Gateway 的 `agent` RPC 方法。当客户端（通道适配器、Web 控制台或 TUI）需要 AI 回复时，它通过 WebSocket 发送如下请求：

```
{
  "type": "req",
  "id": "req-001",
  "method": "agent",
  "params": {
    "message": "帮我分析这段代码的性能问题",
    "sessionKey": "agent:main:telegram:default:dm:12345",
    "channel": "telegram",
    "deliver": true,
    "idempotencyKey": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

`agent` RPC 处理器（`src/gateway/server-methods/agent.ts`）是一个**异步双响应**设计：

```
// src/gateway/server-methods/agent.ts（简化版）
agent: async ({ params, respond, context, client }) => {
  // 1. 参数校验
  if (!validateAgentParams(params)) {
    respond(false, undefined, errorShape(...));
    return;
  }

  // 2. 幂等性去重
  const cached = context.dedupe.get(`agent:${idem}`);
  if (cached) {
    respond(cached.ok, cached.payload, cached.error, { cached: true });
    return;
  }

  // 3. 会话解析与投递计划
  const deliveryPlan = resolveAgentDeliveryPlan({ ... });

  // 4. 立即响应"已接受"（第一个 res 帧）
  const accepted = { runId, status: "accepted", acceptedAt: Date.now() };
  context.dedupe.set(`agent:${idem}`, { ts: Date.now(), ok: true, payload: accepted });
  respond(true, accepted, undefined, { runId });

  // 5. 异步执行 Agent 循环（不阻塞 WebSocket）
  void agentCommand({ message, sessionKey, ... })
    .then((result) => {
      // 第二个 res 帧：完成
      respond(true, { runId, status: "ok", result }, undefined, { runId });
    })
    .catch((err) => {
      // 第二个 res 帧：错误
      respond(false, { runId, status: "error", summary: String(err) }, ...);
    });
};
```

> **衍生解释：双响应模式**是 OpenClaw 协议中的一个重要设计。对于长时间运行的操作（如 AI 对话可能持续数十秒），立即返回一个"已接受"响应让客户端知道请求已被处理，随后在操作完成时发送第二个响应帧。这避免了客户端因超时而重复发送请求，同时 `idempotencyKey` 机制确保即使重发也不会重复执行。

15.2.2 步骤 1：参数校验与会话解析

在 `agentCommand`（`src/commands/agent.ts`）里，第一步是参数校验和会话解析：

```
// src/commands/agent.ts（简化版）
export async function agentCommand(opts: AgentCommandOpts) {
  const body = (opts.message ?? "").trim();
  if (!body) throw new Error("Message (--message) is required");

  const cfg = loadConfig();

  // 验证 agentId 是否存在
  if (agentIdOverride) {
    const knownAgents = listAgentIds(cfg);
    if (!knownAgents.includes(agentIdOverride)) {
      throw new Error(`Unknown agent id "${agentIdOverride}".`);
    }
  }

  // 解析会话
  const { sessionId, sessionKey, sessionEntry, storePath, isNewSession } =
    resolveSession({ cfg, to: opts.to, sessionId: opts.sessionId, sessionKey: opts.sess

  // 解析超时
  const timeoutMs = resolveAgentTimeoutMs({ cfg, overrideSeconds: timeoutSecondsRaw });
  // ...
}
```

会话解析（`resolveSession`）的输出决定了 Agent 运行的上下文：

- `sessionId` ——会话的唯一 UUID，对应磁盘上的 JSONL 转录文件
- `sessionKey` ——复合键（如 `agent:main:telegram:default:dm:12345`），唯一标识一个对话上下文
- `sessionEntry` ——会话存储中的元数据记录，包含思考级别、模型覆盖、技能快照等
- `isNewSession` ——是否为全新会话（影响是否需要加载技能快照）

15.2.3 步骤 2：`agentCommand` — 模型解析、技能快照加载

会话解析完成后，`agentCommand` 进入模型解析和技能准备阶段：

模型解析

模型解析按以下优先级确定最终使用的提供者和模型：

```
// src/commands/agent.ts（简化版）
// 1. 读取全局默认模型
const { provider: defaultProvider, model: defaultModel } = resolveConfiguredModelRef({
  cfg: cfgForModelSelection,
  defaultProvider: DEFAULT_PROVIDER,  // "anthropic"
  defaultModel: DEFAULT_MODEL,        // "claude-sonnet-4-20250514"
});

// 2. 检查会话级别的模型覆盖
const storedModelOverride = sessionEntry?.modelOverride;
if (storedModelOverride) {
  // 验证覆盖模型是否在允许列表中
  const key = modelKey(candidateProvider, storedModelOverride);
  if (allowedModelKeys.has(key)) {
    provider = candidateProvider;
    model = storedModelOverride;
  }
}

// 3. 解析思考级别
resolvedThinkLevel = resolveThinkingDefault({ cfg, provider, model, catalog });
```

优先级链为：**会话覆盖 > Agent 配置 > 全局默认**。如果会话中存储了模型覆盖（通过 `/model` 指令设置），且该模型在允许列表中，则使用覆盖的模型。

技能快照加载

对于新会话或技能快照缺失的情况，系统会扫描工作区目录构建技能快照：

```
// src/commands/agent.ts
const needsSkillsSnapshot = isNewSession || !sessionEntry?.skillsSnapshot;
const skillsSnapshot = needsSkillsSnapshot
  ? buildWorkspaceSkillSnapshot(workspaceDir, {
      config: cfg,
      eligibility: { remote: getRemoteSkillEligibility() },
      snapshotVersion: skillsSnapshotVersion,
      skillFilter: resolveAgentSkillsFilter(cfg, sessionAgentId),
    })
  : sessionEntry?.skillsSnapshot;
```

技能快照（`SkillSnapshot`）记录了 Agent 可用的所有技能及其当前状态。`skillFilter` 参数支持按 Agent 过滤技能——多 Agent 场景下，不同 Agent 可以只看到自己的技能子集。

15.2.4 步骤 3：`runEmbeddedPiAgent` — 队列序列化、auth profile 解析、Pi 会话构建

这是 Agent Loop 中最核心的步骤。`runEmbeddedPiAgent` 函数（`src/agents/pi-embedded-runner/run.ts`）执行以下关键操作：

队列序列化

函数的第一件事是把任务排入两级队列：

```
// src/agents/pi-embedded-runner/run.ts
export async function runEmbeddedPiAgent(params) {
  const sessionLane = resolveSessionLane(params.sessionKey);  // "session:agent:main:..
  const globalLane = resolveGlobalLane(params.lane);           // "main" 或 "subagent"

  return enqueueSession(() =>          // 第一级：会话车道（序列化）
    enqueueGlobal(async () => {        // 第二级：全局车道（限流）
      // ... 核心逻辑 ...
    }),
  );
}
```

两级队列的含义：

- **会话车道**——同一会话的请求必须串行执行（避免并发修改同一对话历史）
- **全局车道**——限制全系统的 LLM 并发调用数量（避免超出 API 速率限制）

Auth Profile 解析

每次 LLM 调用都需要有效的 API 密鑰。OpenClaw 支持多个 Auth Profile（认证配置），并按优先级选择：

```
// src/agents/pi-embedded-runner/run.ts（简化版）
const profileOrder = resolveAuthProfileOrder({
  cfg: params.config,
  store: authStore,
  provider,
  preferredProfile: preferredProfileId,
});

// 跳过处于冷却期的 Profile
while (profileIndex < profileCandidates.length) {
  const candidate = profileCandidates[profileIndex];
  if (candidate && isProfileInCooldown(authStore, candidate)) {
    profileIndex += 1;
    continue;
  }
  await applyApiKeyInfo(candidate);
  break;
}
```

> **衍生解释**：**冷却期**（Cooldown）是一种速率限制恢复机制。当某个 Auth Profile 因为速率限制、认证失败或计费错误被标记为失败后，系统会让它进入一段冷却期，期间不会再尝试使用该 Profile。冷却期过后自动恢复。这避免了反复使用已知失败的凭据。

上下文窗口保护

在发送请求之前，系统会检查模型的上下文窗口是否足够：

```
// src/agents/pi-embedded-runner/run.ts
const ctxInfo = resolveContextWindowInfo({
  cfg: params.config,
  provider,
  modelId,
  modelContextWindow: model.contextWindow,
  defaultTokens: DEFAULT_CONTEXT_TOKENS,
});
const ctxGuard = evaluateContextWindowGuard({
  info: ctxInfo,
  warnBelowTokens: CONTEXT_WINDOW_WARN_BELOW_TOKENS,
  hardMinTokens: CONTEXT_WINDOW_HARD_MIN_TOKENS,
});
if (ctxGuard.shouldBlock) {
  throw new FailoverError(
    `Model context window too small (${ctxGuard.tokens} tokens).`,
    { reason: "unknown", provider, model: modelId }
  );
}
```

核心循环：尝试 → 错误恢复 → 重试

`runEmbeddedPiAgent` 的主体是一个 `while (true)` 循环，实现了多层错误恢复：

```
// src/agents/pi-embedded-runner/run.ts（简化版）
while (true) {
  // 执行一次 LLM 调用
  const attempt = await runEmbeddedAttempt({ ... });
  const { aborted, promptError, timedOut } = attempt;

  if (promptError && !aborted) {
    // 错误恢复分支
    if (isContextOverflowError(errorText)) {
      // 尝试自动压缩会话历史
      if (overflowCompactionAttempts < MAX_OVERFLOW_COMPACTION_ATTEMPTS) {
        await compactEmbeddedPiSessionDirect({ ... });
        continue;  // 压缩后重试
      }
      return { payloads: [{ text: "Context overflow...", isError: true }], ... };
    }

    if (isFailoverErrorMessage(errorText) && await advanceAuthProfile()) {
      continue;  // 切换 Auth Profile 后重试
    }

    const fallbackThinking = pickFallbackThinkingLevel({ message: errorText, attempted
    if (fallbackThinking) {
      thinkLevel = fallbackThinking;
      continue;  // 降低思考级别后重试
    }
  }

  // 成功完成
  if (lastProfileId) {
    await markAuthProfileGood({ store, provider, profileId: lastProfileId });
  }
  return { payloads, meta: { durationMs, agentMeta } };
}
```

错误恢复策略按优先级依次尝试：

顺序	错误类型	恢复策略
1	上下文溢出	自动压缩会话历史（最多 3 次）
2	认证/速率限制错误	切换到下一个 Auth Profile

顺序	错误类型	恢复策略
3	思考级别不支持	降级思考级别（如 xhigh → high）
4	超时	切换 Auth Profile（可能是速率限制导致的超时）
5	其他可故障转移错误	抛出 FailoverError，由外层的模型回退机制处理

15.2.5 步骤 4： `subscribeEmbeddedPiSession` — 事件桥接

在 `runEmbeddedAttempt` 内部，Pi Agent 的事件会被桥接到 Gateway 的事件系统。事件桥接将 Pi Agent 的底层事件转换为 Gateway 可以广播的高级事件：

```
// 事件桥接示意（概念层面）
Pi Agent 事件                    Gateway 事件
─────────────                    ────────────
tool.start                  →    chat { stream: "tool", phase: "start" }
tool.result                 →    chat { stream: "tool", phase: "result" }
assistant.delta              →    chat { stream: "assistant", delta: "..." }
assistant.message_end       →    chat { stream: "assistant", phase: "end" }
lifecycle.start             →    chat { stream: "lifecycle", phase: "start" }
lifecycle.end               →    chat { stream: "lifecycle", phase: "end" }
```

这些桥接事件通过 Gateway 的 WebSocket 广播机制实时推送给所有订阅的客户端，使得：

- **Web 控制台**可以实时显示 AI 的逐字输出（流式 delta）
- **TUI 终端**可以显示工具调用的进度
- **原生应用**可以更新消息气泡的状态

活跃运行注册表（ `runs.ts` ）管理了所有当前正在执行的 Agent 运行：

```
// src/agents/pi-embedded-runner/runs.ts
const ACTIVE_EMBEDDED_RUNS = new Map<string, EmbeddedPiQueueHandle>();

export function setActiveEmbeddedRun(sessionId: string, handle: EmbeddedPiQueueHandle)
  ACTIVE_EMBEDDED_RUNS.set(sessionId, handle);
}

export function isEmbeddedPiRunActive(sessionId: string): boolean {
  return ACTIVE_EMBEDDED_RUNS.has(sessionId);
}

export function queueEmbeddedPiMessage(sessionId: string, text: string): boolean {
  const handle = ACTIVE_EMBEDDED_RUNS.get(sessionId);
  if (!handle || !handle.isStreaming()) return false;
  void handle.queueMessage(text);
  return true;
}
```

`queueEmbeddedPiMessage` 是**消息转向**（Steer）机制的核心——如果 Agent 正在流式输出中，新消息可以被注入到当前的对话循环中，而不是排队等待。

15.2.6 步骤 5：结果汇总、用量统计、会话持久化

当 Agent Loop 完成后（无论成功还是失败），`agentCommand` 执行收尾工作：

生命周期事件发射

```
// src/commands/agent.ts
if (!lifecycleEnded) {
  emitAgentEvent({
    runId,
    stream: "lifecycle",
    data: {
      phase: "end",
      startedAt,
      endedAt: Date.now(),
      aborted: result.meta.aborted ?? false,
    },
  });
}
```

生命周期事件的 `end` 阶段触发多个下游操作：子 Agent 注册表的完成检测、 `agent.wait` 的等待者通知等。

会话存储更新

```
// src/commands/agent.ts
if (sessionStore && sessionKey) {
  await updateSessionStoreAfterAgentRun({
    cfg,
    sessionId,
    sessionKey,
    storePath,
    sessionStore,
    defaultProvider: provider,
    defaultModel: model,
    fallbackProvider,
    fallbackModel,
    result,
  });
}
```

会话存储更新包括：

- **Token 用量**——累加输入/输出 Token 数
- **模型信息**——记录实际使用的提供者和模型（可能经过故障转移后与请求的不同）
- **会话 ID**——确保会话 ID 一致性

结果投递

```
// src/commands/agent.ts
return await deliverAgentCommandResult({
  cfg,
  deps,
  runtime,
  opts,
  sessionEntry,
  result,
  payloads,
});
```

`deliverAgentCommandResult` 将 Agent 的回复文本通过配置的通道发送给用户。如果 Agent 已经通过消息工具（如 Telegram 工具、Slack 工具）直接发送了消息， `didSendViaMessagingTool` 标志会抑制重复投递。

模型回退的封装

整个 `runEmbeddedPiAgent` 调用被包裹在 `runWithModelFallback` 中：

```
// src/commands/agent.ts
const fallbackResult = await runWithModelFallback({
  cfg,
  provider,
  model,
  agentDir,
  fallbacksOverride: resolveAgentModelFallbacksOverride(cfg, sessionAgentId),
  run: (providerOverride, modelOverride) => {
    return runEmbeddedPiAgent({ ...params, provider: providerOverride, model: modelOver
  },
});
```

如果主模型抛出 `FailoverError` ， `runWithModelFallback` 会依次尝试配置的回退模型，形成**三级故障转移链**：

```
主模型（如 anthropic/claude-sonnet-4-20250514）
  ↓ FailoverError
回退模型 1（如 openai/gpt-4o）
  ↓ FailoverError
回退模型 2（如 google/gemini-2.5-pro）
  ↓ FailoverError
最终报错
```

本节小结

1. **Agent Loop 从 `agent` RPC 方法开始**，采用异步双响应模式——立即返回"已接受"，后台异步执行并在完成时发送第二个响应。
2. **参数校验和会话解析**确定了运行上下文，包括会话 ID、会话键、超时设置和技能快照。
3. **模型解析**按"会话覆盖 > Agent 配置 > 全局默认"的优先级确定使用的模型。

4. **`runEmbeddedPiAgent`** 是核心执行函数，实现了两级队列序列化（会话 + 全局）、Auth Profile 轮换和多层错误恢复。
5. **事件桥接**将 Pi Agent 的底层事件转换为 Gateway 广播事件，支持客户端实时展示流式输出。
6. **收尾阶段**包括生命周期事件发射、Token 用量统计、会话持久化和结果投递，外层的模型回退机制提供了额外的容错保障。